How to Get Started

This is a short walk through the `pyrulearn` package for inductive rule learning: load your own data from a CSV, fit a native rule learner, inspect the rules it found in a few different formats, and compare two learners across a handful of the benchmark datasets. See the README for the fuller picture – every learner, interface and demo.

1 Load data from a CSV

`titanic.csv` next to this notebook is the classic Titanic dataset – one row per passenger, `survived` the target. A plain look at it first, the way you'd look at any CSV:

[1]:
```
import pandas as pd

df = pd.read_csv("titanic.csv")
df.head()
```

[1]:

	pclass	survived	sex	age	sibsp	parch	fare	embarked
0	1	survived	female	29.0000	0	0	211.3375	S
1	1	survived	male	0.9167	1	2	151.5500	S
2	1	died	female	2.0000	1	2	151.5500	S
3	1	died	male	30.0000	1	2	151.5500	S
4	1	died	female	25.0000	1	2	151.5500	S

`pyrulearn.data.io.read_csv` reads the same file into a `DataRepresentation`: it guesses each column's type – numeric, ordinal (a numeric column with only a few distinct values, such as `pclass`, `sibsp` and `parch` here) or nominal –, discretizes the numeric ones, and expands everything into binary features, ready to fit on. A numeric attribute gets both `x>=t` and `x<t` for every threshold, an ordinal one `x=v`, `x<=v` and `x>v`; nominal and ordinal attributes also get their explicit negations `x!=v`. How those features are *stored* is a separate choice: by default a `VerticalBitmapRepresentation`, the fastest for this package's learners; we ask for the classic `HorizontalBitmapRepresentation` below (`representation=`) because its storage is the plain 0/1 matrix – see "Internal representation".

Numeric columns are split by a discretizer, by default `TreeDiscretizer(max_intervals=8)` (supervised, decision-tree-based thresholds, at most `max_intervals` intervals): more intervals give finer-grained thresholds (more binary features, slower, more prone to overfitting on small data), fewer give coarser ones. The default is 8; we use 3 here.

[2]:
```
from pyrulearn.data import HorizontalBitmapRepresentation, TreeDiscretizer
from pyrulearn.data.io import read_csv
```

```
data = read_csv("titanic.csv", target="survived",␣
 ↪discretizer=TreeDiscretizer(max_intervals=3),
               representation=HorizontalBitmapRepresentation)
print(data.spec.n_features,
      "binary features from",
      len(df.columns) - 1,
      "raw columns")
```

```
82 binary features from 7 raw columns
```

2 Internal representation

`read_csv` (and every other loader) builds a `VerticalBitmapRepresentation` unless told otherwise – the 0/1 matrix packed by feature, one bit-vector over the examples per feature, on which the native learners here fit fastest (see `demos/representations_report.md`). The default comes from `pyrulearn.data.REPRESENTATION_PREFERENCE_ORDER`. We asked for `HorizontalBitmapRepresentation` above because its storage *is* the 0/1 matrix learners fit on, row by row: `data.X`, with `data.spec` the `DataSpec` describing what each column means (which raw attribute, which test). The other representations – `VerticalBitmapRepresentation`, `NListRepresentation` (a prefix tree of the rows' true features) and `SparseDataRepresentation` (`scipy` sparse matrices) – build `data.X` on demand. All of them are equivalent: same rules, same predictions, whichever a learner gets; only speed and memory differ.

Every feature's name, in full:

[3]:
```
import textwrap

print(textwrap.fill(", ".join(data.spec.feature_names), width=80))
```

```
pclass=1, pclass=2, pclass=3, pclass<=1, pclass<=2, pclass>1, pclass>2,
pclass!=1, pclass!=2, pclass!=3, sex=female, sex=male, age>=8.5, age>=60.2,
age<8.5, age<60.2, sibsp=0, sibsp=1, sibsp=2, sibsp=3, sibsp=4, sibsp=5,
sibsp=8, sibsp<=0, sibsp<=1, sibsp<=2, sibsp<=3, sibsp<=4, sibsp<=5, sibsp>0,
sibsp>1, sibsp>2, sibsp>3, sibsp>4, sibsp>5, sibsp!=0, sibsp!=1, sibsp!=2,
sibsp!=3, sibsp!=4, sibsp!=5, sibsp!=8, parch=0, parch=1, parch=2, parch=3,
parch=4, parch=5, parch=6, parch=9, parch<=0, parch<=1, parch<=2, parch<=3,
parch<=4, parch<=5, parch<=6, parch>0, parch>1, parch>2, parch>3, parch>4,
parch>5, parch>6, parch!=0, parch!=1, parch!=2, parch!=3, parch!=4, parch!=5,
parch!=6, parch!=9, fare>=15.2, fare>=76.0, fare<15.2, fare<76.0, embarked=C,
embarked=Q, embarked=S, embarked!=C, embarked!=Q, embarked!=S
```

Which features a column gets depends on the type `read_csv` inferred for it:

[4]:
```
for name, attr in data.spec.attributes.items():
    features = [f.name for f in data.spec.feature_specs if f.attribute == name]
    print(f"{name:9s} {attr.type.name.lower():8s} {len(features):2d} features: "
          f"{', '.join(features[:5])}{', ...' if len(features) > 5 else ''}")
```

```
pclass    ordinal  10 features: pclass=1, pclass=2, pclass=3, pclass<=1,
```

```
pclass<=2, …
sex       binary    2 features: sex=female, sex=male
age       numeric   4 features: age>=8.5, age>=60.2, age<8.5, age<60.2
sibsp     ordinal  26 features: sibsp=0, sibsp=1, sibsp=2, sibsp=3, sibsp=4, …
parch     ordinal  30 features: parch=0, parch=1, parch=2, parch=3, parch=4, …
fare      numeric   4 features: fare>=15.2, fare>=76.0, fare<15.2, fare<76.0
embarked  nominal   6 features: embarked=C, embarked=Q, embarked=S, embarked!=C,
embarked!=Q, …
```

- `pclass`, `sibsp` and `parch` are numbers, but with only a few distinct values each (at most `max_ordinal_values=10`, the default), so `read_csv` makes them **ordinal**: they keep their order, and every value can be tested on its own – `pclass=1`, `pclass<=2`, `pclass>1`, and with negation `pclass!=3`.
- `age` and `fare` have many distinct values, so they are **numeric**: the discretizer chooses a few thresholds (here at most 3 intervals, so at most 2 thresholds), and each threshold `t` gives the two features `x>=t` and `x<t`. There are no `=` features, as single values of a continuous quantity are rarely worth testing.
- `sex` has exactly two values, so it is **binary**: `sex=female` and `sex=male`, each the negation of the other.
- `embarked` is text with three values, so it is **nominal**: `embarked=v` for every value, and with negation `embarked!=v`.

A peek at a few rows and columns (the full matrix is much bigger):

[5]:
```
import pandas as pd
from IPython.display import display

df_bool = pd.DataFrame(data.X, columns=data.spec.feature_names)
with pd.option_context("display.max_columns", 8, "display.max_rows", 6):
    display(df_bool)
```

```
      pclass=1  pclass=2  pclass=3  pclass<=1  …  embarked=S  embarked!=C  \
0         True     False     False       True  …        True         True
1         True     False     False       True  …        True         True
2         True     False     False       True  …        True         True
…          …        …         …          …    …         …            …
1306     False     False      True      False  …       False        False
1307     False     False      True      False  …       False        False
1308     False     False      True      False  …        True         True

      embarked!=Q  embarked!=S
0            True        False
1            True        False
2            True        False
…             …           …
1306         True         True
1307         True         True
1308         True        False
```

```
[1309 rows x 82 columns]
```

3 Fit a rule learner

`pyrulearn` features several native re-implementations of well-known rule-learning algorithms, as well as interfaces to their original implementations. They all can be called on a uniform data representation, and return different types of rule-based models (rule sets, lists, etc.) in a uniform representation.

`Pypper` is a version of the classic RIPPER algorithm. Every learner fits the same way: `fit(data)`.

Pypper holds out a random third of the training data to prune its rules, so the rules depend on the seed (`random_state`, 0 by default); we use seed 9 here, which gives a model of three rules.

[6]:
```
from pyrulearn.learners.seco import Pypper

model = Pypper(random_state=9).fit(data)
```

4 Inspect the rules

Prolog is `to_string()`'s default format; the rules (and the whole model) also render in a few other formats via `to_string(fmt=...)`, useful for different purposes, from reading them as logic to a compact positional encoding. (`ascii=True` sticks to plain ASCII connectives – `to_string()`'s default uses unicode connectives instead.)

[7]:
```
# prolog, pretty-printed: easier to scan for rules with many conditions
print(model.to_string(pretty=True))
```

```
survived(X) :-  % (233/17)
    pclass(X, V1), V1 <= 2,
    sex(X, female).
survived(X) :-  % (139/34)
    sex(X, female),
    embarked(X, V1), V1 \= S.
survived(X) :-  % (25/1)
    pclass(X, V1), V1 <= 2,
    age(X, V2), V2 < 8.5.
% default
died(X) :- true.  % (809/500)
```

Each rule's trailing `% (tp/fp)` is its own count: how many training rows its *body alone* covers that are (`tp`) or aren't (`fp`) its target – computed in isolation, independently of the other rules, not of what it actually decides once earlier rules in the list have already claimed some rows. The default rule above shows this starkly: its body is empty (`true`), so its count is over *all* 1309 training rows – 809 died, 500 survived – not just the ones that reach it after the rules above it are done.

[8]:
```
# readable logic: "if ... then ..."
print(model.to_string(fmt="logic", ascii=True))
```

```
if   pclass <= 2 AND sex = female -> survived  % (233/17)
elif sex = female AND embarked != S -> survived  % (139/34)
elif pclass <= 2 AND age < 8.5 -> survived  % (25/1)
else -> died  % (809/500)
```

[9]:
```
# bare conditions, no connective or head
print(model.to_string(fmt="conditions", ascii=True))
```

```
survived: pclass <= 2, sex = female  % (233/17)
survived: sex = female, embarked != S  % (139/34)
survived: pclass <= 2, age < 8.5  % (25/1)
died:  % (809/500 - default)
```

[10]:
```
# positional 1/0 pattern, one token per feature, 1 where the rule has
# that condition -- pretty=True shows the full vector (otherwise, with
# many features, it truncates)
print(model.to_string(fmt="pattern", pretty=True))
```

```
survived: 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0  % (233/17)
survived: 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1  % (139/34)
survived: 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0  % (25/1)
died:  % (809/500 - default)
```

Small aggregate stats – rule count, length, coverage:

[11]:
```
from pyrulearn.evaluation import summarize

summarize(model, data)
```

[11]:
```
{'n_rules': 3,
 'mean_length': 2.0,
 'median_length': 2.0,
 'rules_per_target': {'survived': 3},
 'mean_n_covered': 149.66666666666666}
```

5 Without negated features

The negated features (`x!=v`) can be switched off with `include_negations=False` – numeric attributes keep both `x>=t` and `x<t`, which test each threshold from both sides. The same learner with the same seed on the same data:

[12]:
```
```

```
data_pos = read_csv("titanic.csv", target="survived",␣
  ↪discretizer=TreeDiscretizer(max_intervals=3),
                    representation=HorizontalBitmapRepresentation,␣
  ↪include_negations=False)
model_pos = Pypper(random_state=9).fit(data_pos)
print(data_pos.spec.n_features, "binary features")
print(model_pos.to_string(pretty=True))
```

```
61 binary features
survived(X) :-  % (233/17)
    pclass(X, V1), V1 <= 2,
    sex(X, female).
survived(X) :-  % (102/11)
    sex(X, female),
    embarked(X, C).
survived(X) :-  % (37/20)
    sex(X, female),
    parch(X, 0),
    embarked(X, Q).
survived(X) :-  % (44/8)
    age(X, V1), V1 < 8.5,
    sibsp(X, V2), V2 <= 2.
% default
died(X) :- true.  % (809/500)
```

The first rule is the same. The second one, `sex=female` and `embarked!=S`, can't be stated without `!=`: Pypper has to spell it out as one rule for each of the other two ports, `embarked=C` and `embarked=Q` – the latter with an extra condition – so the model gets one rule longer. Negated features thus often give shorter and more readable rules, at the cost of more features to search.

6 Compare two learners across a few datasets

`pyrulearn` can access about a hundred small, medium, and large benchmark datasets through its `Catalog`, fetched automatically through OpenML (100 of its 104 entries; the rest are pointer-only, to datasets found elsewhere).

[13]:
```
from pyrulearn.experiments.catalog import Catalog

catalog = Catalog.default()
print(len(catalog.entries), "datasets, e.g.:")
for e in catalog.entries[:5]:
    print(f"{e.name:28s} {e.task:10s} {e.size:6s} {e.n_instances:>6} rows")
```

```
104 datasets, e.g.:
adult                        binary     large   48842 rows
analcatdata_authorship       multiclass small     841 rows
analcatdata_dmft             multiclass small     797 rows
anneal                       multiclass small     898 rows
```

```
audiology                      multiclass small     226 rows
```

`run_cv` handles the cross-validation, binarization per fold, timing and failure-handling for you, discretizing each fold the same way `read_csv` did above, via its own `discretizer=`. Here's `Pypper` against `PFossil` (another native separate-and-conquer learner) on three small datasets:

[14]:
```
from pyrulearn.evaluation import format_accuracy, format_mean_count
from pyrulearn.experiments.runner import run_cv
from pyrulearn.learners.seco import PFossil

datasets = catalog.select(names=["vote", "tic-tac-toe", "hepatitis"])
results = run_cv([Pypper(), PFossil()], datasets, n_folds=5,␣
 ↪discretizer=TreeDiscretizer(max_intervals=8), verbose=False)

def formatted(table):
    """Accuracies as percentages, rule counts with one decimal -- as in the␣
 ↪reports."""
    out = table.copy()
    out["accuracy"] = table["accuracy"].map(format_accuracy)
    out["n_rules"] = table["n_rules"].map(format_mean_count)
    return out.round({"fit_time": 3, "mean_rank": 2})

formatted(results.groupby(["dataset", "learner"])[["accuracy", "n_rules",␣
 ↪"fit_time"]].mean())
```

[14]:
dataset	learner	accuracy	n_rules	fit_time
hepatitis	PFossil	83.87%	14.0	0.009
	Pypper	76.13%	1.8	0.008
tic-tac-toe	PFossil	97.39%	14.8	0.015
	Pypper	98.33%	8.0	0.033
vote	PFossil	95.63%	8.6	0.012
	Pypper	96.32%	2.6	0.333

Summary by learner – mean across datasets and folds, plus mean rank (1 = best) – the same summary every comparison demo in `demos/` reports:

[15]:
```
from pyrulearn.experiments.stats import mean_rank

summary = results.groupby("learner")[["accuracy", "n_rules", "fit_time"]].mean()
summary["mean_rank"] = mean_rank(results, "accuracy")
formatted(summary)
```

[15]:
learner	accuracy	n_rules	fit_time	mean_rank
PFossil	92.30%	12.5	0.012	1.67
Pypper	90.26%	4.1	0.125	1.33

7 Where to go from here

- `README.md` – a detailed description of the package, with a summary of all provided functionalities, including a list of all native and interfaced learners.
- `demos/` – demos of different functionalities. Each can be called in a quick mode (default), which demonstrates its base functionality, and a full mode, which does a more elaborate investigation of an issue. Sample outputs are enclosed in the checked-in reports.